

# Multimodal AI & Vision Transformers

*How images become token sequences — and how text and vision share one embedding space*

**25 min**

Duration

**B3 – Apply**

Bloom's

**L3 – Intermediate**

Level

**Topics 1–3**

Builds on

# Learning Objectives

*By the end of this session, you will be able to:*

## B2 – Explain

The core insight of multimodal AI: any modality can become a token sequence

## B3 – Derive

The shape arithmetic:  $224 \times 224$  image,  $16 \times 16$  patches  $\rightarrow$  196 tokens + CLS = 197

## B3 – Describe

The end-to-end VLM pipeline: ViT  $\rightarrow$  projection  $\rightarrow$  LLM decoder

## B3 – Describe

How ViT divides an image into patches and produces a sequence of embeddings

## B3 – Explain

How a projection layer aligns vision embeddings with the text embedding space

## B3 – Explain

CLIP contrastive training and what a shared image-text embedding space enables

# Session Roadmap

5 sub-topics — from the core insight to the shared embedding space

## 4.1 What Multimodal AI Is — The Core Insight

*Modality conversion, not invention*

## 4.2 Vision Transformer (ViT) — Images as Token Sequences

*Patch → embed → attend*

## 4.3 Image Embeddings and the Projection Bridge

*Aligning vision and text spaces*

## 4.4 VLM Architecture — How It All Connects

*ViT → Projection → LLM end-to-end*

## 4.5 CLIP and Contrastive Learning

*Shared image-text embedding space*

Assignments A4 (ViT patch decomposition) & A5 (CLIP similarity matrix) introduced inline | Audio/video & diffusion deferred to M4.2 & M4.3

# 4.1

## What Multimodal AI Is — The Core Insight

*Any modality can become a token sequence. The Transformer runs unchanged.*

## The Core Insight — Modality Conversion, Not Invention

*The Transformer from Topic 3 does not change. What changes is everything upstream of it.*

Text-only models operate over discrete tokens. The insight of multimodal AI: images, audio, and video can all be converted into sequences of vectors in the same  $d_{\text{model}}$ -dimensional space that text embeddings occupy. Once that conversion exists, the Transformer runs unchanged.

**Text****Word tokens**Token embedding lookup →  $d_{\text{model}}$  vector**Image****Patch tokens (ViT)**Patch projection →  $d_{\text{model}}$  →  $d_{\text{model}}$  vector**Audio****Spectrogram frames**Conv + projection →  $d_{\text{model}}$  →  $d_{\text{model}}$  vector**Video****Frame-level patches**Spatial + temporal embed →  $d_{\text{model}}$  vector

All modalities land in the same  $d_{\text{model}}$ -dimensional space. The Transformer then processes the combined sequence with ordinary self-attention. This session covers images. The same principle extends to audio, video, and beyond.

## Multimodal Models in Production

*The same unified-token-sequence principle across all major systems*

|                                                                                       |                                                                                                                                   |                                         |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <b>GPT-4o (OpenAI)</b><br><i>Text + Image + Audio</i>                                 | Unified token stream of all three modalities processed through a single Transformer. Native audio — not speech-to-text then text. | Production since May 2024               |
| <b>Gemini 1.5 Pro (Google DeepMind)</b><br><i>Text + Image + Audio + Video + Code</i> | 1M+ token context window. Processes entire documents, hour-long videos, and mixed-modality inputs in one pass.                    | MoE Transformer architecture            |
| <b>LLaVA-1.6 (Open-weight)</b><br><i>Text + Image</i>                                 | Llama 3 backbone + ViT vision encoder + linear projection layer. Fully open-weight. The reference architecture for this session.  | Most downloaded open VLM on HuggingFace |
| <b>Claude 3 (Anthropic)</b><br><i>Text + Image + Document</i>                         | Custom vision encoder. Handles PDFs, diagrams, charts, and natural images alongside text in the same context window.              | Production since March 2024             |

Common architecture pattern in all of the above: vision encoder + projection layer + language model decoder. This session builds that pattern piece by piece.

# 4.2

## Vision Transformer (ViT)

*Images as Token Sequences — patch, project, attend*

## ViT — Converting an Image into a Token Sequence

Five steps: divide, flatten, project, prepend CLS, add positional embedding

①

### Divide into patches

Split the input image into a grid of non-overlapping fixed-size patches. Original ViT uses 16×16 pixels per patch on a 224×224 image.

$$224 \div 16 = 14 \text{ patches per side} \rightarrow 14 \times 14 = 196 \text{ patches}$$

②

### Flatten each patch

Each 16×16 patch has 256 pixels × 3 colour channels (RGB) = 768 raw values. Flatten the 2D spatial grid into a 1D vector.

Patch shape: (16, 16, 3) → flatten → vector of length 768

③

### Linear projection to d\_model

Multiply each flattened patch vector by a learned weight matrix E (the patch embedding projection). Maps 768 → d\_model (e.g. 768 or 1024).

$$z_i = \text{patch}_i \cdot E \quad \text{where } E \in \mathbb{R}^{(768 \times d_{\text{model}})}$$

④

### Prepend [CLS] token

Insert a special learnable [CLS] embedding at position 0. After the Transformer processes the sequence, this vector aggregates information from all 196 patches via self-attention.

Sequence: [ CLS, z\_1, z\_2, ..., z\_196 ] length = 197

⑤

### Add 2D positional embeddings

Add learned positional embeddings encoding the row and column of each patch on the image grid. Without these, the model cannot distinguish a patch at top-left from one at bottom-right.

$$\text{input}_i = z_i + \text{pos\_embed}(\text{row}_i, \text{col}_i)$$

After step ⑤: 197 vectors each of dimension d\_model enter a standard Transformer encoder from Topic 3. No convolution. No architectural change.

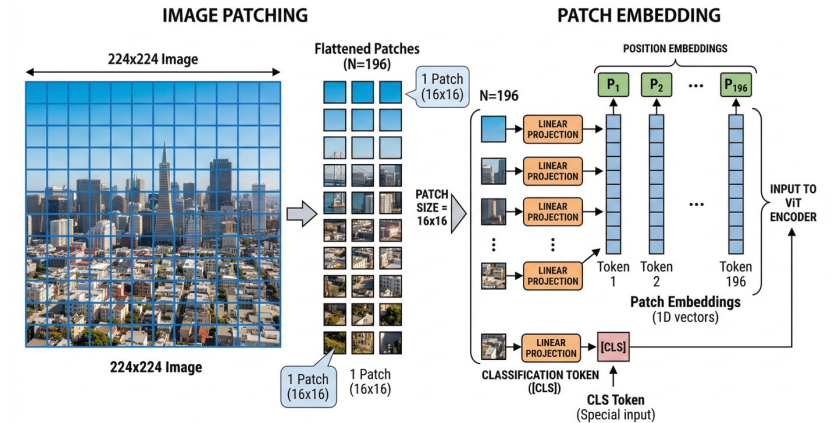


# Shape Arithmetic — From Pixels to Tokens

Every number is traceable. No magic steps.

Input:  $224 \times 224$  RGB image | Patch size:  $16 \times 16$  |  $d_{\text{model}}$ : 768

|                             |                            |                                                                             |
|-----------------------------|----------------------------|-----------------------------------------------------------------------------|
| Image tensor                | <b>(224, 224, 3)</b>       | Height $\times$ Width $\times$ RGB channels                                 |
| Patch grid                  | <b>(14, 14, 16, 16, 3)</b> | 14 rows $\times$ 14 cols of $16 \times 16 \times 3$ patches                 |
| After flatten               | <b>(196, 768)</b>          | 196 patches, each flattened to $16 \times 16 \times 3 = 768$ values         |
| After projection $\times E$ | <b>(196, 768)</b>          | Each patch vector mapped to $d_{\text{model}} = 768$ via learned matrix $E$ |
| After prepend [CLS]         | <b>(197, 768)</b>          | CLS token inserted at position 0 — one extra vector                         |
| After positional embed      | <b>(197, 768)</b>          | Positional vectors added element-wise — shape unchanged                     |
| Transformer input           | <b>(197, 768)</b>          | 197 tokens $\times$ 768 dimensions — processed by standard self-attention   |



# Assignment A4 — Manual Patch Decomposition with NumPy

*Make the 196-token intuition completely concrete before moving on*

## WHAT · WHY · EXPECTED OUTCOME

### WHAT

Given a 64×64 grayscale image loaded as a NumPy array, write the patch extraction pipeline from scratch:

1. Reshape the image into a patch grid
2. Flatten each patch to a 1D vector
3. Print the tensor shape at every step
4. Verify final shape = (16, 256) for 8×8 patches on a 64×64 image

### WHY

The ViT pipeline only becomes intuitive when you do the reshape by hand.

Printing shapes at each step forces you to trace exactly how a 2D image becomes a 1D sequence of vectors — the fundamental operation underlying all multimodal AI.

### EXPECTED OUTCOME

A notebook showing the shape trace:  
(64,64) → (8,8,8,8) → (64,64) → (16,256)

A one-sentence answer to: why does the ViT need positional embeddings? What spatial information is lost during the flatten step?

Bonus: repeat for 16×16 patches on a 224×224 image and verify 196 tokens.

`import numpy as np` | `img.reshape(n_h, p_h, n_w, p_w)` | Submit notebook before mentor session

# 4.3

## Image Embeddings and the Projection Bridge

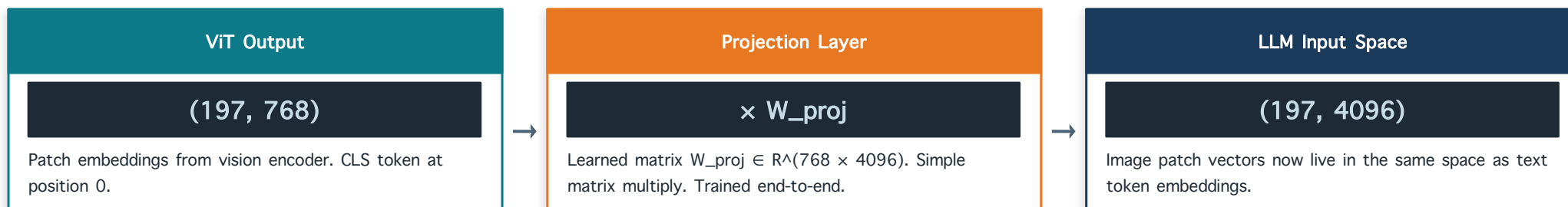
*Aligning the vision space with the text space — the missing link in VLMs*

## The Projection Bridge — Aligning Vision and Text Spaces

Raw ViT output and LLM text embeddings live in different spaces. One matrix fixes this.

The problem: a ViT trained as a vision encoder produces embeddings in its own learned space (e.g. 768-dim). The language model was trained with text token embeddings in its own space (e.g. 4096-dim for Llama 3 8B). These two spaces are not compatible — their dimensions may differ and their learned geometries are unrelated.

The solution: a learned linear projection matrix  $W_{\text{proj}}$  maps from the vision space into the LLM text embedding space



Combined token sequence fed to LLM:  $[ \text{img\_patch}_1 \dots \text{img\_patch}_{196} \mid \text{CLS} \mid \text{text}_1 \dots \text{text}_n ]$

CLS token role: After the Transformer processes the full 197-token sequence, the CLS vector at position 0 has attended to all 196 patch tokens. It acts as a compressed whole-image summary. For classification tasks, only the CLS vector is passed to the output head. For VLMs, all 196 patch vectors are used so the LLM can attend to specific image regions.

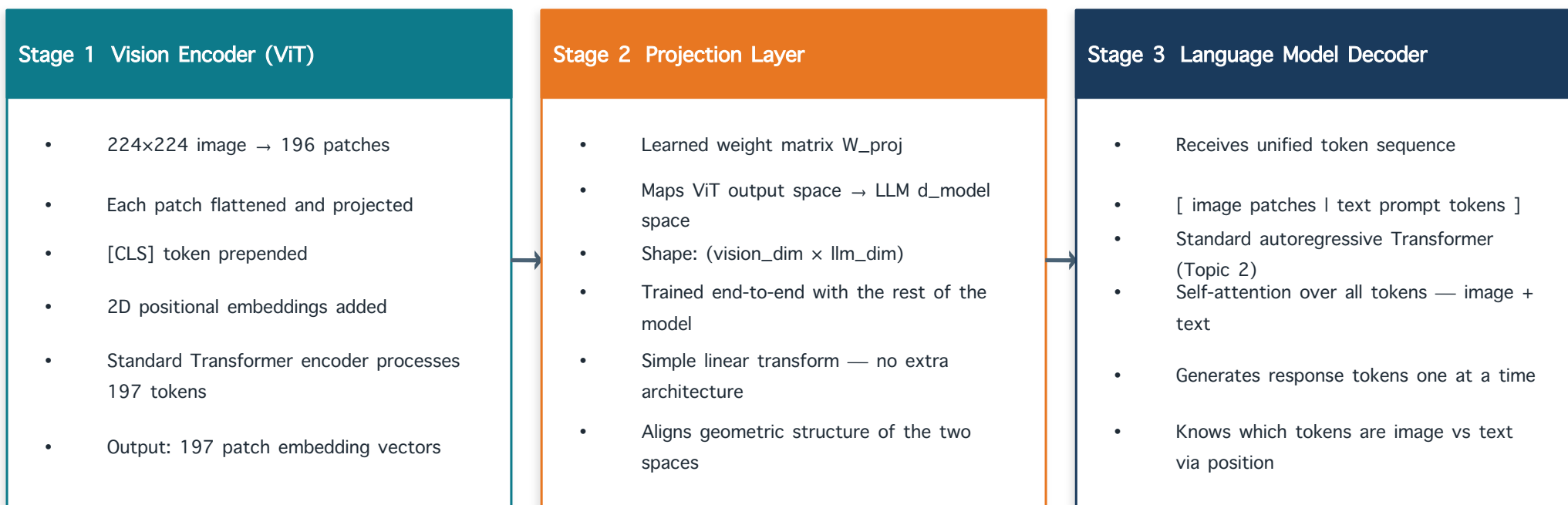
# 4.4

## VLM Architecture — How It All Connects

*ViT → Projection → LLM Decoder — end-to-end data flow*

## End-to-End VLM Architecture

Three stages. The LLM from Topics 1–3 is stage 3 — almost unchanged.



**Key insight:** the LLM in stage 3 is almost identical to what you know from Topics 1–3. The innovation lives in stages 1 and 2.

## Worked Example — Answering "What is in this image?"

Tracing the data flow from pixels to generated text

Input: [photo of Eiffel Tower] + prompt: "Describe this image in one sentence."

- ViT processes image**  
196 patch embeddings of the tower, sky, surroundings. CLS token aggregates whole-image context.
- W\_proj aligns spaces**  
Each ViT vector projected into LLM's embedding space. Now geometrically compatible with text tokens.
- Text tokens appended**  
Prompt tokenised: 'Describe this image in one sentence.' = ~12 tokens. Concatenated after image tokens.
- LLM attends over all 209 tokens**  
Self-attention lets every text token see every image patch. The model grounds its generation in visual evidence.
- Output generated**  
'The Eiffel Tower stands against a clear blue sky in Paris, France.'  
— each token drawn from the shared embedding space.

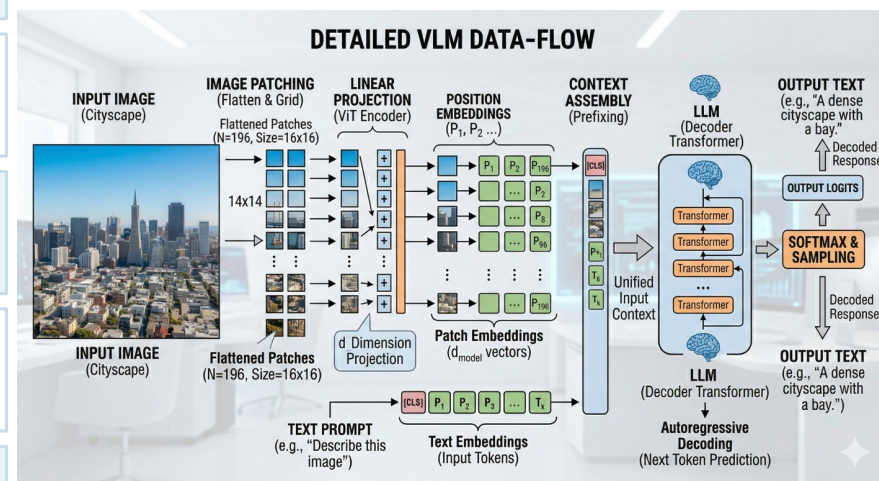
197 tokens

$197 \times 4096$

$197 + 12 = 209$

Autoregressive

Token by token



LLaVA: Llama 3 8B + ViT-L/14 + linear projection / GPT-4V: proprietary ViT + GPT-4 / Claude 3: custom vision encoder + Claude backbone

# 4.5

## CLIP and Contrastive Learning

*Creating the shared image-text embedding space from scratch*



## CLIP — Contrastive Language-Image Pre-training

*Radford et al. (2021) · 400 million image-caption pairs · Two encoders, one shared space*

### Image Encoder (ViT)

Input: raw image

Converts image into a single embedding vector via ViT patch decomposition + CLS pooling.

Output: one vector in shared embedding space (e.g. 512-dim).

→ SHARED  
SPACE ←

### Text Encoder (Transformer)

Input: text caption / class name

Converts text into a single embedding vector via Transformer encoder + [EOS] pooling.

Output: one vector in shared embedding space (same 512-dim as image encoder).

### Training objective (contrastive loss — InfoNCE):

#### MAXIMISE

Cosine similarity of the  $N$  matching image-caption pairs in the batch  
"A photo of a cat" should be close to the image of a cat

#### MINIMISE

Cosine similarity of all  $N^2 - N$  non-matching pairs  
"A photo of a cat" should be far from images of cars, buildings, food

Training data: 400 million image-caption pairs scraped from the web. No manual labels. Self-supervised via the image-text co-occurrence signal.

## The CLIP Similarity Matrix

For a batch of  $N$  pairs, CLIP trains to produce a diagonal-dominant  $N \times N$  cosine similarity matrix

Batch of 4 image-caption pairs — what CLIP's cosine similarity matrix should look like after training:

|              | A cat on a mat | Eiffel Tower | A golden retriever | Ocean sunset |
|--------------|----------------|--------------|--------------------|--------------|
| Cat image    | 0.92✓          | 0.05         | 0.12               | 0.08         |
| Tower image  | 0.06           | 0.94✓        | 0.03               | 0.11         |
| Dog image    | 0.14           | 0.04         | 0.91✓              | 0.09         |
| Sunset image | 0.07           | 0.09         | 0.06               | 0.95✓        |

High diagonal values ( $>0.90$ ) = matched pairs. Low off-diagonal values ( $<0.15$ ) = non-matched pairs. This is exactly what the contrastive loss optimises.

## What the Shared Embedding Space Enables

*Once images and text live in the same space, distance = semantic similarity*

### Zero-Shot Image Classification

How: Encode the image. Encode each candidate class name as text (e.g. 'a photo of a cat', 'a photo of a car'). The class whose text embedding is closest to the image embedding wins.

*Why it works: No training labels needed for new classes. Just describe the class in text.*

ImageNet zero-shot: 76.2% top-1 accuracy — competitive with supervised ResNet-50 trained on all labels.

### Image-Text Retrieval

How: Given a text query, encode it. Compute cosine similarity against a database of pre-encoded image embeddings. Return the top-k most similar images.

*Why it works: Powers reverse image search, visual search in e-commerce, and media asset retrieval.*

'Find all photos of sunsets on a beach' — retrieves semantically matching images with no keyword tags.

### Vision Encoder for Downstream VLMs

How: Use the pre-trained CLIP image encoder as the ViT stage in a VLM. The encoder already understands image-text alignment from contrastive pre-training.

*Why it works: Dramatically reduces training cost. The projection layer only needs to align two already-semantically-rich spaces.*

DALL-E 2 uses CLIP image embeddings as the conditioning signal. LLaVA uses CLIP ViT-L/14 as its vision backbone.

CLIP was published by Radford et al. (OpenAI, 2021). Pre-trained on 400M pairs. Model weights are open — available on HuggingFace.

# Assignment A5 — CLIP Cosine Similarity Matrix

*Observe the shared embedding space with your own hands*

## WHAT · WHY · EXPECTED OUTCOME

### WHAT

Load the CLIP model from HuggingFace (openai/clip-vit-base-patch32).

Encode 3 images of your choice (e.g. a cat, the Eiffel Tower, a beach).

Encode 3 matching text descriptions.

Compute the 3×3 cosine similarity matrix between all image-text pairs.

Identify which pairs have the highest similarity.

### WHY

The cosine similarity matrix makes the shared embedding space tangible.

You will see with your own numbers that matched image-text pairs score 0.85+ while non-matched pairs score 0.1–0.3.

This is the exact contrastive training objective playing out on real data.

### EXPECTED OUTCOME

A 3×3 similarity matrix (printed as a table or heatmap).

A short paragraph (4–6 sentences) answering:  
— What does the contrastive training objective optimise?

— Why does a high diagonal score mean the training worked?

— Name one production use case enabled by this shared space.

Bonus: try one deliberately wrong text pair (e.g. 'a photo of fire' for a beach image).

```
from transformers import CLIPProcessor, CLIPModel | model.get_image_features() & model.get_text_features() | F.cosine_similarity()
```

## Four Misconceptions — Cleared

*Common errors at this stage of the multimodal curriculum*

M1

**MYTH: "VLMs understand images the way humans do"**

**CORRECTION:** A VLM converts an image into 196 patch embedding vectors and processes them with self-attention. It does not perceive objects, depth, or scene structure the way the human visual cortex does. Its apparent understanding emerges from statistical patterns learned across millions of image-caption pairs.

M2

**MYTH: "The LLM sees the raw image pixels"**

**CORRECTION:** The LLM never sees pixels. It receives a sequence of  $d_{\text{model}}$ -dimensional vectors produced by the ViT vision encoder and the projection layer. From the LLM's perspective these are just more tokens — no different in format from text token embeddings.

M3

**MYTH: "CLIP teaches the model to classify images"**

**CORRECTION:** CLIP is not trained with class labels. It is trained with a contrastive objective on image-caption pairs with no manual annotation. Classification is an emergent capability — the shared embedding space enables it via cosine similarity to text class names.

M4

**MYTH: "Adding more patch tokens always gives better results"**

**CORRECTION:** More patches increase the sequence length fed to the LLM, which increases attention computation quadratically. There is a hard tradeoff between visual resolution and compute. Production models use pooling, dynamic resolution, or tiling strategies to manage this.

## References & Further Reading

Foundational papers, tools, and resources for Topic 4

### Foundational Papers

- [1]** Dosovitskiy et al. (2020). An Image is Worth 16x16 Words: ViT. ICLR 2021.  
*Original Vision Transformer — patches, CLS token, positional embeddings*
- [2]** Radford et al. (2021). Learning Transferable Visual Models from Natural Language Supervision. (CLIP)  
*Contrastive pre-training on 400M pairs — defines the shared embedding space*
- [3]** Liu et al. (2023). Visual Instruction Tuning. (LLaVA). NeurIPS.  
*Open-weight VLM reference architecture — ViT + linear projection + Llama*
- [4]** Alayrac et al. (2022). Flamingo: a Visual Language Model for Few-Shot Learning. NeurIPS.  
*Gated cross-attention VLM — interleaves vision into a frozen language model*
- [5]** OpenAI (2023). GPT-4 Technical Report.  
*GPT-4V multimodal capabilities — production-scale VLM reference*

### Tools & Demos

- Model** openai/clip-vit-base-patch32 on HuggingFace  
[huggingface.co/openai/clip-vit-base-patch32](https://huggingface.co/openai/clip-vit-base-patch32)
- Model** LLaVA-1.6 (llava-hf/llava-v1.6-mistral-7b-hf)  
[huggingface.co/llava-hf](https://huggingface.co/llava-hf)
- Tool** CLIP Playground — OpenAI interactive demo  
[clip-as-service.jina.ai](https://clip-as-service.jina.ai)
- Blog** Jay Alammar — The Illustrated CLIP  
[jalammar.github.io](https://jalammar.github.io)
- Paper** Dosovitskiy ViT — original patch diagram  
[arxiv.org/abs/2010.11929](https://arxiv.org/abs/2010.11929)
- Video** Yannic Kilcher — CLIP paper explained  
[youtube.com/@YannicKilcher](https://youtube.com/@YannicKilcher)

## Topic 4 Summary — Multimodal AI & Vision Transformers

*Six essential takeaways*

1

### Modality conversion, not invention

Images, audio, and video are converted into  $d_{\text{model}}$ -dimensional vectors. Once in that space, the Transformer from Topics 1–3 runs unchanged on the combined token sequence.

2

### ViT: $224 \times 224 \rightarrow 197$ tokens

Divide into  $16 \times 16$  patches  $\rightarrow$  flatten  $\rightarrow$  project to  $d_{\text{model}}$   $\rightarrow$  prepend [CLS]  $\rightarrow$  add 2D positional embeddings  $\rightarrow$  standard Transformer encoder. Shape:  $(197, d_{\text{model}})$ .

3

### The projection bridge aligns two spaces

A learned linear matrix  $W_{\text{proj}}$  maps ViT output dimensions into the LLM's  $d_{\text{model}}$  space. This is the only new component between the vision encoder and the language model.

4

### VLM = ViT + projection + LLM decoder

Three stages. The LLM is almost identical to Topics 1–3. The LLM sees image patch vectors and text token vectors as one flat sequence and attends over all of them.

5

### CLIP creates the shared embedding space

Contrastive training on 400M image-caption pairs. Maximise similarity of matched pairs, minimise for non-matched. Result: images and text live in the same geometric space.

6

### CLS token = compressed image summary

After Transformer processing, the [CLS] vector has attended to all 196 patches. Used for classification (single vector) or as a whole-image signal when full patch resolution is not needed.

## M1–M4 Section 1 — Full Consolidation Picture

*All four topics complete. What you now hold and where it leads.*

| Topic 1<br>Completed                                                                                                                                                                                                    | Topic 2<br>Completed                                                                                                                                                                                                                     | Topic 3<br>Completed                                                                                                                                                                                                       | Topic 4<br>Completed                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>✓ Token vocabulary</li> <li>✓ Softmax + logits</li> <li>✓ Temperature &amp; sampling</li> <li>✓ Cross-entropy loss</li> <li>✓ Perplexity</li> <li>✓ Connection to RAG</li> </ul> | <ul style="list-style-type: none"> <li>✓ Autoregressive loop</li> <li>✓ Stopping conditions</li> <li>✓ Repetition failure</li> <li>✓ 3 hallucination types</li> <li>✓ Confidence vs truth</li> <li>✓ 3 misconceptions cleared</li> </ul> | <ul style="list-style-type: none"> <li>✓ RNN bottleneck</li> <li>✓ Q / K / V mechanism</li> <li>✓ Attention formula</li> <li>✓ Causal masking</li> <li>✓ Multi-head attention</li> <li>✓ Full Transformer block</li> </ul> | <ul style="list-style-type: none"> <li>✓ Modality conversion</li> <li>✓ ViT patch tokens</li> <li>✓ Projection bridge</li> <li>✓ VLM architecture</li> <li>✓ CLIP contrastive</li> <li>✓ Shared embed space</li> </ul> |

This consolidation session covers the foundation of M1–M4 Sec 1. Next: mentor-led discussion sessions on Assignments A1–A5.



## What We Deferred — and Where It Lands

*Each item has a more appropriate module. None are lost.*

**M4.2 Audio & Speech Models****Audio modality — Whisper, spectrograms**

*Spectrogram tokenisation and Whisper encoder-decoder architecture — warrants its own session*

**M4.3 Multimodal Fusion****Video tokens and temporal modelling**

*Frame-level ViT + temporal attention — covered in the multimodal fusion strategies session*

**M4 Generative Vision****Diffusion models (DALL-E 3, Stable Diffusion)**

*Denoising diffusion process is architecturally distinct — CLIP role mentioned, derivation deferred*

**M4.3 Training Multimodal Systems****InfoNCE contrastive loss derivation**

*The loss function math — described at the objective level here, derived in the training session*

**M4.3 Multimodal Fine-Tuning****LLaVA two-stage training procedure**

*Pre-training projection layer, then instruction tuning — covered in the fine-tuning session*

**M4.4 Real-World Multimodal Apps****BLIP-2, Flamingo, Qwen-VL variants**

*Architecture variants and production trade-offs — more appropriate in the applied session*

# Assignments A1–A5 — Full Overview

*All five assignments from the consolidation lecture*

Submit all five notebooks before the mentor-led group discussion session. One week from today.

A1

Topic 1

## Logprobs & Softmax

Call API with logprobs=True. Temperature sweep  $T=0/0.7/1.5$ . Comparison table + 3-sentence softmax explanation.

A2

Topic 2

## Autoregressive Loop

API with stream=True. Log each token. Count forward passes. One sentence on why 200-token costs  $2\times$  a 100-token response.

A3

Topic 3

## BertViz Attention

Two sentences with ambiguous pronoun 'it'. Two attention heatmaps. Written analysis: which head resolved coreference and why.

A4

Topic 4

## NumPy Patch Decomposition

Manual patch extraction on a  $64\times 64$  image. Print shape at every step. One sentence on why positional embeddings are needed.

A5

Topic 4

## CLIP Similarity Matrix

Load CLIP, encode 3 images + 3 texts. Compute  $3\times 3$  cosine similarity matrix. Paragraph on what contrastive training achieves.

Consolidation Lecture Complete

# M1–M4 Section 1

## Foundation Established

*Upcoming in the programme:*

### Mentor Sessions

#### A1–A5 Discussion (10 mentors, groups of 5–8)

One week from today. Mentors will review submissions and run group discussions. Bring your notebooks.

### M3 APIs Recap

#### Context Window, KV Cache, Latency & Cost

Now that the full Transformer block is clear, the API-level implications of context length and KV cache are covered properly.

### M4 Continues

#### M4.2 Audio · M4.3 Multimodal Fusion · M4.4 Applications

Audio modality (Whisper), multimodal fusion strategies, fine-tuning VLMs, and real-world application architectures.

Assignments A1–A5 due before mentor session · All decks available on the course portal · Questions to your assigned mentor